

Krishi Mitra — Prototype Assessment & Ideal Prototype Gap List

Current Prototype Score: 7.5 / 10

Why 7.5?

Strength	+Points
Working end-to-end RAG pipeline	+2.0
Semantic chunking (embedding-based boundaries)	+1.0
Hybrid retrieval (dense + sparse + BM25 + RRF)	+1.0
Numeric faithfulness safety layer	+1.0
Empirical threshold justification	+0.5
Cross-judge capability (OpenRouter)	+0.5
Clean code (no dead code, no hardcoded strings)	+0.5
Category-aware retrieval boost	+0.5
Indic NLP normalization	+0.5
Base total	7.5

What's costing us 2.5 points:

Gap	-Points	Why It Hurts
No query condensation (multi-turn broken)	-0.8	Examiners WILL type a follow-up and watch it fail
No streaming (blocking 30s+ responses)	-0.5	Perceived latency is bad in a live demo
n=16 eval set (too thin)	-0.5	“Only 16 questions?” is an easy examiner critique
No GitHub Action / CI signal	-0.3	One-hour task that shows engineering maturity
No Docker (defense-day risk)	-0.2	“Works on my machine” is a real risk during demo
No query expansion	-0.2	Simple LLM call that would improve retrieval on vague queries

Ideal Prototype: What's Missing (Do These Now)

These are **NOT** in your current TODO list (or are under-prioritized). They are all achievable within **1–3 days** and would push your score to **9/10**.

Tier 1: Demo-Killers (Fix These First)

#	Gap	Effort	Defense Impact	In Your TODO?
1	Query condensation for multi-turn	4–6 hrs	HIGH —Examiners will test follow-ups. “?” currently retrieves nothing.	#14 (yes, but needs action)
2	Streaming responses	3–4 hrs	HIGH —30s of silence looks broken. Streaming makes it feel alive.	#15 (yes, but needs action)
3	Re-label gold.jsonl after semantic chunking	2–3 hrs	HIGH —All your eval numbers are garbage until this is done.	Not explicitly listed
4	Expand eval set to n=40–50	4–5 hrs	HIGH —n=16 is statistically thin. 40+ questions from KCC dataset is defensible.	#18 (yes, but needs action)

Tier 2: Quick Wins (High Signal, Low Effort)

#	Gap	Effort	Defense Impact	In Your TODO?
5	GitHub Action running re-fusal_test.py on push	1 hr	MEDIUM —One .github/workflows/ci.yml file. Shows you understand CI.	#12 (yes, but needs action)
6	Docker Compose (backend + frontend + MongoDB + Qdrant)	2–3 hrs	MEDIUM —Eliminates “works on my machine” risk on defense day.	#9 (yes, but needs action)
7	Lightweight query expansion	2 hrs	MEDIUM —LLM rewrites vague queries (“?”) into specific ones (“ ”) before retrieval.	NOT in TODO
8	Adversarial test suite (beyond refusal)	2 hrs	MEDIUM —Test numeric transposition (10g→100g), off-topic injection, empty queries.	NOT in TODO

Table 4 – continued

#	Gap	Effort	Defense Impact	In Your TODO?
9	Document tempera- ture=0.0 as design choice	30 min	LOW —One paragraph in report. Shows intentional trade-off.	#39 (yes, report writing)
10	Document PII limi- tation	15 min	LOW —One sentence in report.	#41 (yes, report writing)
11	Production Readiness section in report	30 min	LOW —List auth, rate limiting, logging as deliberately deferred.	#42 (yes, report writing)

Tier 3: Nice-to-Have (Would Impress)

#	Gap	Effort	Defense Impact	In Your TODO?
12	Confidence badges in frontend	2–3 hrs	MEDIUM —Show search_score and accuracy_score visually. Already wired in backend.	Already done per architect review
13	Source display in frontend	2–3 hrs	MEDIUM —Show which chunks were retrieved. Already wired in backend.	Already done per architect review
14	Simple query analytics log	1 hr	LOW —Log query + category + score to a file. Shows you think about improvement loops.	NOT in TODO
15	Pre-flight check script	1 hr	LOW —Script that verifies .env, MongoDB, Qdrant, and model availability before starting.	NOT in TODO

Future Production Scope (Mention in Report, Don't Build)

These are **genuinely out of scope** for a 2-week thesis prototype. List them in a “Production Readiness” or “Future Work” section to show you understand the delta.

Infrastructure & Ops

Item	Why It's Future Scope
Authentication (OAuth, API keys)	Farmer-facing prototype doesn't need auth yet

Table 6 – continued

Item	Why It' s Future Scope
Rate limiting (per-user, per-IP)	Free tier + prototype scale makes this unnecessary
Load balancing / horizontal scaling	Single-process FastAPI is fine for demo
Structured logging (JSON, ELK, Datadog)	<code>print()</code> is sufficient for thesis debugging
Request tracing (OpenTelemetry)	Needed at scale, overkill for n=1 examiner
Metrics dashboard (Grafana)	No real traffic to monitor
Secrets manager (AWS Secrets Manager)	<code>.env</code> is acceptable for prototype
Automated backups	Qdrant file can be copied manually

Model & Data

Item	Why It' s Future Scope
Domain fine-tuning of BGE-M3	Requires GPU + labeled data + weeks of training
Fine-tuning a Kannada LLM	Architect review explicitly says “don’ t attempt this”
Training a custom tokenizer	BGE-M3’ s SentencePiece is sufficient for prototype
Multi-crop expansion (ragi, tomato)	Requires new data collection and re-labeling
Real-time data ingestion	Static JSON corpus is sufficient for demo
Incremental index updates	Full rebuild is fine for a 40-chunk corpus
Embedding versioning	Only one model version in use
Embedding caching (Redis)	Re-embedding is fast enough at prototype scale

UX & Features

Item	Why It' s Future Scope
Voice input/output (STT/TTS)	Complex integration, not core to RAG thesis
Speech-to-speech (KLLM-style)	Research project in itself, not a 2-week addition
Mobile app	Web UI is sufficient for demo

Table 8 – continued

Item	Why It' s Future Scope
Offline mode	Requires model quantization + edge deployment
Real-time market price integration	Needs external API + rate limits + caching
Farmer feedback loop (thumbs up/down)	Requires user base + analytics infrastructure
Push notifications	Needs mobile app + FCM/APNs integration
Multi-language support (Hindi, Tamil)	Kannada focus is defensible for Karnataka thesis

Advanced RAG

Item	Why It' s Future Scope
Claim-level NLI verification	Complex —needs a separate NLI model per claim
Citation extraction (source attribution)	Requires structured output + chunk mapping
Parent-document retrieval	Would help but adds complexity; semantic chunking is sufficient
Hierarchical chunking (summary → detail)	Overkill for a 40-section corpus
Graph RAG (entity extraction + knowledge graph)	Research-level, not prototype
Agentic RAG (tool use, planning)	Far beyond thesis scope
Self-correction loops	Requires multiple LLM calls per query
Active learning from user feedback	Needs sustained user traffic

Recommended 2-Week Sprint Plan

Week 1: Core Fixes (Push to 8.5/10)

Day	Task	Effort	Output
1	Test new Groq model, smoke test LLM client	2 hrs	Working model confirmed
1-2	Rebuild database with semantic chunking, re-label gold.jsonl	4 hrs	New chunk IDs + fresh gold labels

Table 10 – continued

Day	Task	Effort	Output
2–3	Query condensation for multi-turn	6 hrs	Follow-up queries work end-to-end
3–4	Streaming responses (backend SSE + frontend)	6 hrs	Live demo feels responsive
4–5	Expand eval set to n=40–50 from KCC dataset	6 hrs	Statistically defensible eval
5	Run full ablation + threshold sweep with new data	3 hrs	Updated numbers for report

Week 2: Polish & Report (Push to 9/10)

Day	Task	Effort	Output
6	GitHub Action for refusal_test.py	2 hrs	CI badge in repo
6–7	Docker Compose setup	4 hrs	One-command startup
7–8	Lightweight query expansion	3 hrs	Vague queries retrieve better
8–9	Adversarial test suite	3 hrs	Numeric transposition + injection tests
9–10	Write report sections: Design Choices, Production Readiness, Future Work	6 hrs	Complete report draft
10	Pre-flight check script + final integration test	3 hrs	Defense-day confidence

The Honest Bottom Line

You are at 7.5/10. With 2 focused weeks, you can reach **9/10** by fixing the demo-killers (multi-turn, streaming, eval size) and adding quick wins (CI, Docker, query expansion).

The remaining 1 point to 10/10 requires production infrastructure that is deliberately out of scope — and that’s fine. A 9/10 prototype with a clear “Future Work” section is a stronger defense than a 10/10 production system that you half-built and can’t demo reliably.

Your biggest risk: An examiner types a follow-up question and watches it fail. Fix query condensation first. Everything else is secondary.